

AI Application Engineering

A 16-week, project-first learning plan for backend engineers moving into LLM, RAG, agentic-AI, and AI-automation roles

Purpose

This is a living reference plan. It focuses on becoming credible for AI Application Engineer, GenAI Backend Engineer, AI Automation Engineer, and Backend Engineer—AI/LLM focus roles. The plan turns existing backend, Docker, API, monitoring, and database experience into production-oriented AI engineering evidence.

***Core rule:** do not treat completion of the IBM certificate as the finish line. Every concept must become a test, metric, architecture decision, documented trade-off, or working feature in one of the two portfolio projects.*

Operating model

Weekly allocation	Hours	Purpose
Structured learning	3 h	IBM modules plus the primary official source for that topic.
Independent implementation	4 h	Build the same idea independently in the relevant portfolio repository.
Tests, evaluation, refactoring	1.5 h	Convert lessons into tests, metrics, and cleaner implementation.
Interview drills and notes	1 h	Answer questions in your own words; add a short learning-log entry.
README / portfolio update	0.5 h	Document what changed, a decision, and one known limitation.

Mental-model loop

For every technology, answer: **(1)** What problem does it solve? **(2)** What is the simplest implementation without a framework? **(3)** What does the framework abstract? **(4)** How can it fail in production? **(5)** How will it be tested, observed, and improved?

Target stack

Go deep: Python, Pydantic, pytest, FastAPI, one LLM API, structured outputs, tool calling, RAG, PostgreSQL/pgvector, Redis, LangGraph, MCP, Langfuse, Docker/Compose, GitHub Actions, and one cloud provider.

Know at working awareness: LangChain, LlamaIndex, Chroma, FAISS, CrewAI, AG2/AutoGen, BeeAI, Hugging Face, PyTorch basics, LoRA, vLLM, Kubernetes, Temporal, multimodal models.

Defer: training foundation models, elaborate multi-agent systems, custom vector databases, GPU infrastructure, and fine-tuning projects. First demonstrate that you can build a reliable AI application.

Sixteen-week schedule

The IBM specialization is the backbone; the independent builds are non-negotiable. Course 8 is for framework comparison—not a reason to build a multi-agent demo in every framework.

Weeks 0–1: Production Python baseline

IBM: No IBM module yet: establish the engineering base.

Learn: Official Python docs; FastAPI tutorial/testing; refresh SQL, Docker Compose, GitHub Actions.

Build: Create **ai-engineering-lab**: typed Python package, Pydantic document model, pytest, ruff, CI, learning_log.md.

Mental-model checkpoint: Dependency isolation; parsing vs validation; unit vs integration test.

Weeks 2–3: LLM foundations and structured outputs

IBM: IBM Course 1: *Develop Generative AI Applications: Get Started*.

Learn: Full Stack LLM Bootcamp; Anthropic prompt tutorial; OpenAI structured outputs and function calling docs.

Build: Build **earnings_extractor**. Direct API only; typed JSON for company, period, metrics, guidance, risks, evidence snippets; 20 adversarial/incomplete cases.

Mental-model checkpoint: Schemas constrain response shape, not truth. Use deterministic code for arithmetic and validation.

Weeks 4–6: RAG from first principles

IBM: IBM Courses 2–4: introductory RAG, vector databases, advanced RAG/retrievers.

Learn: Hugging Face Learn for embeddings; pgvector docs; Supabase pgvector guide.

Build: Build **filing-rag-service**: ingest 2–3 public filings/transcripts; preserve metadata; baseline chunking; Postgres + pgvector; keyword + dense retrieval; citations; 35 QA cases.

Mental-model checkpoint: RAG is retrieval plus generation, not “chat with a PDF.” Measure retrieval separately from answer quality.

Weeks 7–8: Productionize the RAG service

IBM: IBM Course 5 only as time permits: multimodal/OCR/transcription breadth.

Learn: FastAPI tutorials, testing and Docker guides; Docker Compose; GitHub Actions.

Build: FastAPI endpoints for ingest/status/query/health; async ingestion; Postgres/pgvector and Redis; auth stub; structured logs; tests; CI.

Mental-model checkpoint: Ingestion is a pipeline; queries are latency-sensitive. Model APIs are unreliable dependencies requiring timeouts/retries.

Weeks 9–10: Tools and safe workflows

IBM: IBM Course 6: *Fundamentals of Building AI Agents*.

Learn: OpenAI function-calling guide; Anthropic tool-use materials.

Build: Build **financial-research-workflow** with read-only document search, deterministic ratio calculator, metrics lookup, typed schemas, allow lists, audit log, retries and a tool budget.

Mental-model checkpoint: The model proposes a tool call; application code validates authorization and executes it. They are not the same thing.

Weeks 11–12: Agents, LangGraph, and MCP

IBM: IBM Courses 7–9: LangGraph/agent patterns; compare CrewAI/AG2/BeeAI; build agents with MCP.

Learn: LangGraph Essentials then Intro to LangGraph; Hugging Face Agents Units 1–2; official MCP docs.

Build: Implement direct Python workflow, then LangGraph states: interpret, retrieve, calculate, draft, validate, approval, finish. Add a read-only FastMCP server and tests.

Mental-model checkpoint: Workflow = code controls known steps. Agent = model dynamically chooses constrained steps. MCP is an integration standard, not a security control.

Weeks 13–14: Evaluation, security, observability

IBM: Start IBM Course 10 capstone; adapt it to your financial-document system instead of a generic multi-agent demo.

Learn: OpenAI evaluation example; Langfuse tracing docs; Full Stack LLM Ops content.

Build: Create 50–70 case eval dataset; measure evidence hit rate, citation correctness, refusal, schema/tool success, latency/cost. Add Langfuse traces, CI regression gate, threat model.

Mental-model checkpoint: A demo working once proves little. Evals and traces show whether a change reliably improves quality and where it failed.

Weeks 15–16: Deployment, portfolio, interviews

IBM: Finish and claim IBM certificate only after portfolio artifacts are complete.

Learn: AWS or GCP fundamentals; revise Python, SQL, async, API, queue/cache, and system design questions.

Build: Deploy one service with Docker; maintain local Compose setup. Publish README, architecture diagram, evaluation table, threat model, cost assumptions, and 5–7 minute walkthrough.

Mental-model checkpoint: Be able to defend every design choice and one meaningful failure. The project deep dive is the interview asset.

Portfolio deliverables

Project	Must demonstrate	Evidence to publish
1. Filing RAG Service	Ingestion, metadata, chunking, dense + keyword retrieval, citations/refusal, FastAPI, Postgres/pgvector, Docker, tests.	README, architecture diagram, API examples, 35+ QA/eval set, evaluation results, trace screenshots, threat model.
2. Financial Research Workflow	Safe tools, deterministic calculations, LangGraph state/routing, MCP, permissions, approval gate, audit logging, evals.	README, tool schemas, workflow diagram, failure scenarios, test/eval results, 5–7 minute demo.

Interview readiness benchmark

- Build a schema-validated extraction endpoint and explain failure handling.
- Explain and implement RAG end-to-end, including retrieval versus generation failure diagnosis.
- Explain when to use prompting, RAG, long context, and fine-tuning.
- Design a safe tool call with authorization outside the model and clear retry/idempotency rules.
- Explain an agent as a controlled workflow: state, stopping conditions, tool budgets, and human approval.
- Demonstrate a basic MCP server/client and articulate its security limits.
- Show an evaluation dataset, retrieval/answer metrics, traces, and a CI regression check.
- Deploy a Dockerized FastAPI service with secrets handled correctly.
- Present one project failure and how tracing/evaluation detects or limits it.

High-value interview questions

- Design a company annual-report assistant: ingestion, retrieval, citations, access control, evaluation, monitoring, and cost.
- A response is wrong: how do you isolate parsing, chunking, retrieval, prompt, model, tool, or validation failure?
- Why use RAG rather than fine-tuning? When is long context sufficient?
- Why might a deterministic workflow be safer than an agent?
- How do you protect a tool-using system from indirect prompt injection and excessive permissions?
- How do you roll out a prompt/model/retrieval change safely?

Source library

All links below are clickable in this PDF. Use the primary source first; use the secondary sources only where they fill a clear gap.

Source	How to use it
IBM RAG and Agentic AI Professional Certificate	Main 10-course spine: GenAI apps, RAG, vector databases, multimodal apps, agents, LangGraph, multi-agent frameworks, MCP, and capstone.
Full Stack LLM Bootcamp	LLM foundations, prompting, LLMOps, and production application perspective.
Anthropic interactive prompt tutorial	Prompt behavior and systematic prompt design exercises.
Anthropic educational courses	API fundamentals, prompting, evaluations, and tool-use material.
OpenAI Structured Outputs	JSON-schema-constrained model outputs.
OpenAI Function Calling	Designing model-mediated calls to external tools/systems.
OpenAI evaluation example	Runnable evaluation examples for structured output.
FastAPI tutorial	Python API fundamentals.
FastAPI testing	Testing FastAPI applications.
FastAPI with Docker	Containerized service deployment.
Hugging Face learning hub	Embeddings, transformers, and related learning materials.
Hugging Face Agents Course	Agent loop, tools, and framework exposure.
pgvector	Postgres extension for vector similarity search.
Supabase pgvector guide	Practical pgvector usage and embedding storage.
LangGraph Essentials — Python	State, nodes, edges, memory, and a practical workflow.
Introduction to LangGraph — Python	Deeper agent/workflow orchestration course.
Model Context Protocol documentation	Official MCP concepts and getting-started documentation.
Langfuse tracing documentation	Open-source LLM tracing, evaluation, prompt management, and metrics.
Python documentation	Modern Python, typing, async, packaging, and standard library reference.
pytest documentation	Testing practices and fixtures.
Pydantic documentation	Data validation, typed schemas, and settings management.
Docker documentation	Docker and Docker Compose reference.
GitHub Actions documentation	Continuous integration workflows.
AWS Skill Builder	Cloud fundamentals if AWS is selected.
Google Cloud Skills Boost	Cloud fundamentals if GCP is selected.

Version 1.0 — July 2026. Revisit after each milestone, job interview, or job-description review; improve depth where real job requirements expose a gap.